

A Platform based on HLS to implement a generic CNN on an FPGA

Darío Baptista^{1,2,3}, F. Morgado-Dias², Leonel Sousa^{1,3}

¹Instituto Superior Técnico, IST, Lisbon, Portugal

²Madeira Interactive Technologies Institute, M-ITI, Funchal, Portugal.

³Instituto de Engenharia de Sistemas e Computadores, Investigação e Desenvolvimento, INESC-ID, Lisboa, Portugal

Abstract — The fast progress in modern applications based on convolution neural network poses new challenges, such as higher precision and real-time response. On the other hand, advances of Field Programmable Gate Arrays tools allows designs based on High-Level Synthesis, allowing a faster and an easier implementation on hardware of solutions for complex problems. However, a significant amount of time and still some level of hardware design expertise are required to implement a convolution neural network on hardware. To solve this difficulty a platform to emulate a generic parameterizable-based convolution neural network on a programmable gate arrays is developed, giving the freedom to specify the network topology and tune the parameterization. This platform, developed in C language and synthesized through High-Level Synthesis, is prepared to configure a floating-point convolution neural network as a “lego”. This approach became attractive because the designer focuses on the network topology, without in-depth understanding of the underlying hardware a requirement.

Keywords—FPGA, CNN, High-Level Synthesis, Flow Architecture.

I. INTRODUCTION

In the 80s, the first Field Programmable Gate Array (FPGA) was created [1]. In contrast to systems based on general-purpose processors, FPGAs allow to explore spatial parallelism. FPGAs outperform the systems based on processors by eliminating the paradigm of sequential execution. This can be important in Convolution Neural Networks (CNN) applications where parallelism and distributed computing are inherently involved.

The quantity of CNN systems implementations on an FPGA developed so far with the capacity for being generic is quite low. An experiment dated back to the early 90’s with Bell Labs’ ANNA chip (a mixed analog/digital neural-network chip) implemented a fast 2D convolution and matrix-vector multiplication [2]–[4]. In 1996, Cloutier et al. [5] presented a CNN implementation for handwritten character recognition. A parallel processor implemented on an FPGA was used in that work. Farabet et al. [6] proposed a CNN where operations of 2D convolution, 2D pooling, point-wise nonlinear mapping and other operations required to process images were implemented on an FPGA. The general functions such as: management of data transfer from/to an external system; store/retrieve kernels from/to an external memory; and control of the computation to implement the CNN; were executed on a software. In the literature other similar approaches are presented [6]–[10]. In 2015, Zhang et al.[11] presented a float-point CNN implementation taking into account the bandwidth optimization to the external memory in order to make a better utilization of the FPGA resources and achieve a higher performance. However, an integration of non-linear function

and other layers, such as max-pooling layer, were not implemented in that work.

A great effort was made by Xilinx to provide High-level Synthesis (HLS). HLS generates a Register Transfer Level (RTL) design from a the algorithmic level [12]. In spite of the fact that HLS can produce the design from a high language, still significant amounts of time and a level of hardware design knowledge are required to implement a CNN solution on an FPGA. The main contribution of this work is a novel way to solve this problem by providing a platform to emulate a generic parameterizable-based CNN solution on an FPGA. An advantage arises of using this platform: the amount of code to be written by designers is reduced dramatically, which saves time and reduces the risk of mistakes. As result, a designer can easily implement a CNN system on an FPGA without the necessity of in-depth understanding of the underlying hardware. In addition to the main contribution, there are secondary contributions:

1. Hyperbolic tangent, ReLU and Softmax function were implemented on FPGA, without aid of an ordinary processor.
2. Kernels and weights are store on-chip, canceling out the external data access to retrieve kernels and weights.
3. A different architecture for designing a CNN on an FPGA are presented and tested: Flow architecture.
4. A method to save FPGA resources without penalizing the throughput.

II. BACKGROUND

Since the emergence of CNN, variants of CNN topologies have been proposed in the literature[13]–[15], [16], [17], [18] [19]. Their basic components are similar and consist in combining three types of layers: convolution, pooling and fully-connected.

A. Convolution Layer

This layer extracts features from an input matrix. The convolution layer is composed by convolution operations, producing feature maps as output [20]. Each element of that feature map is given by equation 1.

$$C_{i,j,m} = \varphi \left(\sum_{l=0}^{L-1} \sum_{p=0}^{H-1} \sum_{q=0}^{H-1} F_{i+p,j+q,l} K_{p,q,l,m} + b_m \right) \quad (1)$$

where F as the input matrix ($F \in \mathbb{R}^{W \times W \times L}$), K as the kernel ($K \in \mathbb{R}^{H \times H \times L \times M}$), b as the bias and φ is a non-linear function.

B. Max-Pooling Layer

The max-pooling layer down-samples and “smooths”, reducing the artifacts and sharp variations on the signal that comes out usually from the convolution layer [21]. The expression for computing the response of each feature map element is given by equation 2.

$$P_{i,j,l} = \varphi \left(\max_{p \in [0;H[\wedge q \in [0;H[} (F_{i+p,j+q,l}) \right) \quad (2)$$

where F as the input matrix ($F \in \mathbb{R}^{W \times W \times L}$), a $H \times H$ pooling window size and φ a nonlinear function

C. Fully-connected Layer

The classification is performed by this kind of layer. Its elements is identical to a neuron of an artificial neural network (ANN), whose response is given by equation 3.

$$y = \varphi \left(\sum_{i=1}^n x_i w_i + b \right) \quad (3)$$

III. PLATFORM DESCRIPTION

Fig.1 illustrates the flow architecture used for implementing a CNN on hardware. It is possible to visualize how the data flows through the CNN across time. In this architecture, all layers are independent of each other and the flow of data is driven by data.

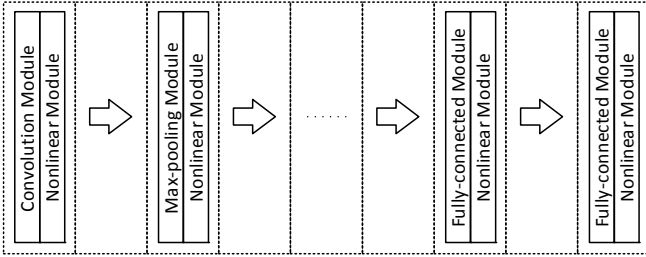


Fig. 1. Illustration of the flow architecture proposed for implementing a deep learning on an FPGA.

After loading the project to Vivado HLS, it is verified that the primary file is written in C code. This file includes a hierarchy of sub-files. One of them is a configuration file which should be written by the designer in order to define the desired CNN. The configuration file gives tools to the designer for configuring the CNN as a “lego”. The designer can specify the topology, enabling the implementation of any CNN which adopts as layers: convolution, pooling and fully-connected. Then, the platform only needs to reorganize “the pieces of lego” to get the CNN described in the configuration file. The interpretation of each layer is composed by two parts. In the first part, a module containing the main operation is placed then, in the second part, a module containing the non-linear function is positioned. Fig. 2 illustrates the interpretation of each layer.

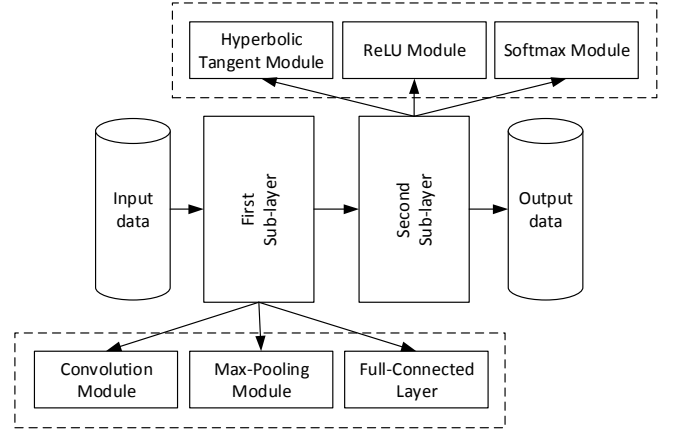


Fig. 2. Illustration of compiler interpretation for each layer

After synthesis is completed, the design is packaged as an intellectual property (IP) block to be used by other tools inside of Vivado Design Suite (e.g. Vivado, System Generator, etc.). The packaged IP includes the Verilog and Hardware Description Language (HDL) design files. Also, report files are generated showing synthesis results. Fig. 3 presents an overview of the process since filling the configuration file by designer until the IP generated by the project synthesis using the Vivado HLS.

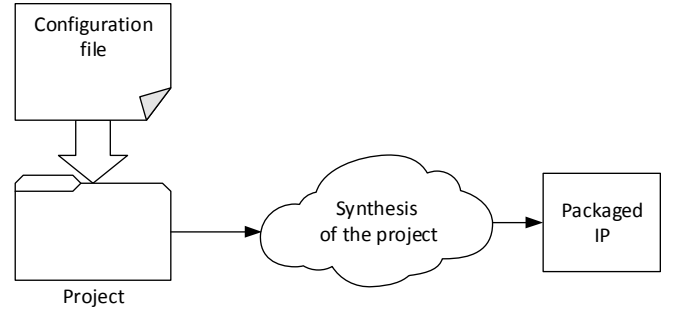


Fig. 3. Diagram illustrating the process since filling the configuration file by designer until the IP block generated by the project synthesis using the Vivado HLS.

IV. DETAILS OF THE HARDWARE IMPLEMENTATION

In the flow architecture, the whole system is seen as a series of input data transformations, where all operations are independent of each other. This architecture lays emphasis on the incremental transformation of data by successive components. Each layer is composed by two parts and the resulting output of each layer is stored in different memories.

A global view of this architecture after synthetizing is illustrated in Fig.4. Analyzing the architecture, pipeline between layers can be implemented. By using a dataflow directive, the pipeline between layers is implemented and, as result, the flow data is optimized. In that case, the data is inserted into the system and then, it is driven by itself through layers. This architecture allows the performance improvement over a statically pipelined solution.

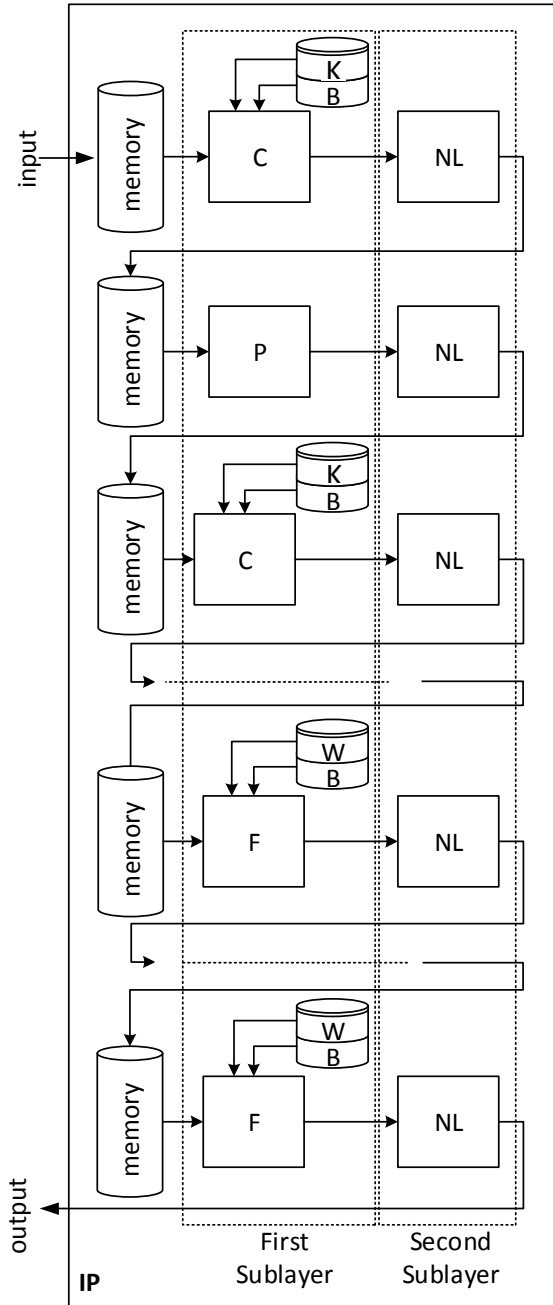


Fig. 4. Illustration of the design interpreted by HLS when the flow architecture is used (C: Convolution Module; P: Max-pooling Module; F: Fully-connected Module; K: Kernel, B: Bias; W: Weight; NL: Non-linear Module).

A. Hyperbolic tangent module

The solution is to implement a strategy to compute the hyperbolic tangent for $x > 0$. The hyperbolic tangent values for $0 \leq x < 6$ are stored using a Look-up-Table (LUT) and, for domain $x > 6$, the hyperbolic tangent is interpolated by $y=1$. Then, for $x < 0$, hyperbolic tangent values are achieved by using the property in equation 4.

$$f(-x) = -f(x) \quad (4)$$

B. ReLU Module

The ReLU does not require complex mathematical operations. Basically, if $x < 0$, then $\varphi(x) = 0$;

otherwise $\varphi(x) = x$. Where $\varphi(x)$ is the output given an input x .

C. Softmax Module

The softmax function is used at the last layer of a CNN-based classifier [22]. This function is given by the equation 5

$$\varphi_{softmax}(x^{(i)}) = \frac{e^{x^{(i)}}}{\sum_{j=0}^k e^{x_k^{(i)}}} \quad (5)$$

To design the exponential function on an FPGA, a hybrid solution was used. This solution consists in decomposing the exponential function into an integer and a fractional part.

$$e^x = e^{int_x} \times e^{frac_x} \quad (6)$$

where int_x is the integer part and $frac_x$ is the fractional part of x . The e^{frac_x} values are computed using a polynomial interpolator and the e^{int_x} values are got through its storage in a ROM [23].

D. Convolution Module

To design this module in HLS, optimization directives are required to search for the best architecture. The code structure after optimization using loop unrolling and pipelining directives is presented in the Pseudo-Code 1.

Pseudo-Code 1. Convolution module

```

N ← layer size; NS ← number of matrix inputs; KR × KC ←
kernel dimension; KO × KO ← output matrix dimension;
for (ro=1 to RO) {
  for (co=1 to CO) {
    //pipeline directive
    for (n=1 to N) {
      //unroll directive
      buf ← bias
      for (ns=1 to NS) {
        //unroll directive
        for (kr=1 to KR) {
          //unroll directive
          for (kc=1 to KC) {
            //unroll directive
            ker ← select the kernel element;
            in ← select the input element;
            temp ← ker * in;
            buf ← buf + temp;
          }
          buf ← buf * k1; //only if a normalization is required
          buf ← buf + k2; //only if a normalization is required
        }
      }
    }
  }
}

```

Hardware pipelining is an optimization technique to exploit the system throughput by overlapping the execution of operations from different loop iterations. Loop unrolling is another technique to exploit parallelism between loop iterations. An advantage of the use of this technique is the significant performance improvement. In contrast, more hardware is used to implement an unrolled loop.

E. Max-pooling Module

The code structure after optimization for loop unrolling and pipelining is presented in the Pseudo-Code 2. Just similar to the previous module, optimization directives are used to enable efficient loop unrolling and hardware pipeline.

Pseudo-Code 2. Max-pooling module

```

N ← layer size; FR × FC ← pooling window dimension; KO × KO ←
output matrix dimension;
#define max (a, b) (((a) > (b)) ? (a) : (b))

for (ro=1 to RO) {
  for (co=1 to CO) {
    //pipeline directive
    for (n=1 to N) {
      //unroll directive
      temp ← 0
      for (fr=1 to FR) {
        //unroll directive
        for (fc=1 to FC) {
          //unroll directive
          in ← select the input element;
          temp ← max(temp, in);
          buf ← temp;
        }
      }
    }
  }
}

```

F. Fully-connected Module

The fully-connected module is used at the end of a CNN. When there are more than one fully-connected layer, every element in the first fully-connected module are connected to all elements in the second fully-connected module. Unrolled loop and hardware pipeline are also used in this module for increasing the utilization of massive computation resources of the FPGA. The code structure is presented in the Pseudo-Code 3.

Pseudo-Code 3. Fully-connected module

```

N ← layer size; NS ← number of matrix inputs; IR × IC ← input
matrix dimension;
for (n=1 to N) {
  //pipeline directive
  buf ← bias
  for (ns=1 to NS) {
    //unroll directive
    for (r=1 to IR) {
      //unroll directive
      for (c=1 to IC) {
        //unroll directive
        wei ← select the weight element;
        in ← select the input element;
        temp ← wei * in;
        buf ← buf + temp;
      }
    }
  }
}

```

V. RESULTS

A Kintex7 KC705 is the FPGA used for the experiences presented in this section

A. Comparison of CNNs using hyperbolic tangent and ReLU

A CNN is composed of several layers connected sequentially. Commonly, each one of these layers is accompanied by a nonlinearity: hyperbolic tangent or ReLU. Both yield strong practical implications. The most widely-used is the ReLU. A model using ReLU makes the training process easier and often achieves better performance. Another benefit of using ReLU is demonstrated in this section. If its implementation on an FPGA is considered, resources can be saved.

For this experience, two CNNs with the same topology but using different non-linear functions in the hidden layer are implemented, as shown in table I. Table II presents the throughput and latency between both CNN.

TABLE I. DESCRIPTION OF TWO CNN USED FOR EXPERIMENTAL ASSESSMENT

	CNN-1	CNN-2
Topology ^a	I[32×32@1]→C[11@5×5 STR 4] → P[11@2×2 STR 2] → C[8@5×5 STR 1] → P[8@2×2 STR 2] → F[20] → F[10]	
non-linearity kind in the hidden layers	ReLU	Hyperbolic Tangent
non-linearity kind in the output layer	Softmax	

^a Image – I [image size @ number of image]; Convolution Layer – C [size layer @kernel window | stride]; Pooling Layer – P [size layer @ pooling window | stride]; Full Layer – F [size layer];

TABLE II. LATENCY AND THROUGHPUT FOR EACH CNN

	CNN-1	CNN-2
Latency (ms)	0.1638	0.1647
Throughput (Samples/s)	13355	

Analyzing the total latency of each CNN, CNN-2 is approximately 1 μ s faster than CNN-1 (about 0.002% faster). The small difference between both is because the hyperbolic tangent requires more time to calculate than the ReLU function. Essentially, there are no changes of throughput between the two CNNs. Although the throughput and the latency is not practically affected, the difference of resources used to implement each CNN is significant. Implementing CNN-1 requires more resources than the CNN-2, as shown in Figure 5 and 6. From these figures, it possible to establish that a CNN using a hyperbolic tangent as non-linearity needs 18.65% more resources than the same CNN using a ReLU as non-linearity.

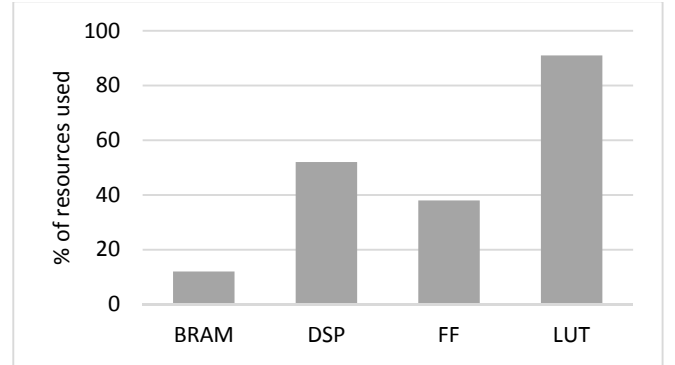


Fig. 5. Resources used by CNN-1. LUT: Look-up Table; FF: Flip-Flops; BRAM: Block of Memory; DSP: Digital Signal Processor;

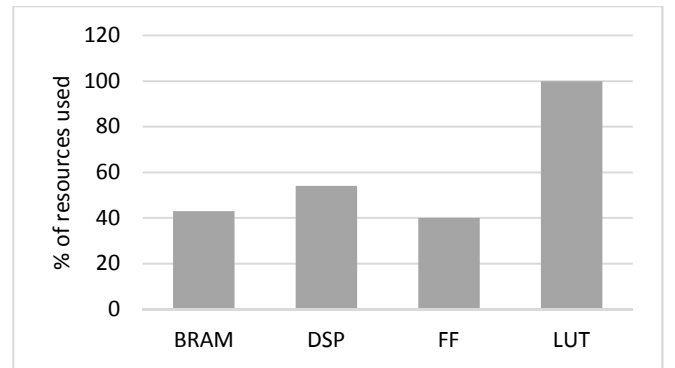


Fig. 6. Resources used by CNN-2. LUT: Look-up Table; FF: Flip-Flops; BRAM: Block of Memory; DSP: Digital Signal Processor;

B. Resources optimization

In this section is discussed how save resources at the expense of the some layer latency without affecting the throughput of the entire CNN. For that, the optimization directives are removed in layers which present lower latency and, as consequence, less resources are used in these layers.

Table III and IV present the latency of four different implementations of CNN-1 and CNN-2, respectively. The shaded fields are the layers where the optimization directives were removed. So, 2nd, 3rd and 4th implementations are increasingly less optimized in terms of latency but use less resources.

TABLE III. THE LATENCY (μ SECONDS) OF FOUR DIFFERENT IMPLEMENTATIONS OF CNN-1

	1 st impl.	2 nd impl.	3 rd impl.	4 th impl.
1 st layer	75.91	75.91	75.91	75.91
2 nd layer	31.75	31.75	31.75	31.75
3 rd layer	45.77	45.77	45.77	45.77
4 th layer	0.36	18.33	18.33	18.33
5 th layer	6.73	17.37	17.37	221.41
6 th layer	2.41	2.41	22.82	22.82

TABLE IV. THE LATENCY (μ SECONDS) OF FOUR DIFFERENT IMPLEMENTATIONS OF CNN-2

	1 st impl.	2 nd impl.	3 rd impl.	4 th impl.
1 st layer	76.09	76.09	76.09	76.09
2 nd layer	31.8	31.81	31.81	31.81
3 rd layer	46.32	46.32	46.32	46.33
4 th layer	0.41	23.45	23.45	26.01
5 th layer	6.79	17.43	17.43	222.81
6 th layer	2.41	2.41	22.82	22.82

Although directives were not withdrawn in the 5th layer latency, there was a small increase in the data processing time in that layer. This happen because the data is not available when it is needed. There is a data dependence produced by the previous layer. In this case the problem is solved by stopping the pipeline in the 5th layer until the flow decision stage is computed. Although this situation provokes an increase of latency in 5th layer and, as consequence, an increase of latency in the entire CNN (Fig. 7). However, the CNN throughput is not affected as shown in the table V. That occurs for the reason that the system accepts new input according to the layer which has the highest latency. Fig. 8 and Fig. 9 present the resources by the 1st implementation and the 3rd implementation using the CNN-1 and CNN-2, respectively.

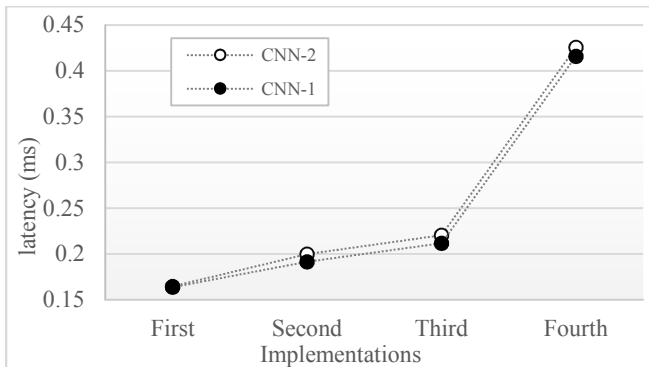


Fig. 7. The CNN-1 and CNN-2 latency usign different implementation.

TABLE V. THE THROUGHPUT (SAMPLES/S) OF CNN-1 AND CNN-2 USIGN DIFFERENT IMPLEMENTATIONS

	1 st impl.	2 nd impl.	3 rd impl.	4 th impl.
CNN-1	13355	13355	13355	4516
	-	↔	↔	↓
CNN-2	13355	13355	13355	4488
	-	↔	↔	↓

Note: ↔: Equal value as previous implementation;
↓: Lower value than previous implementation;

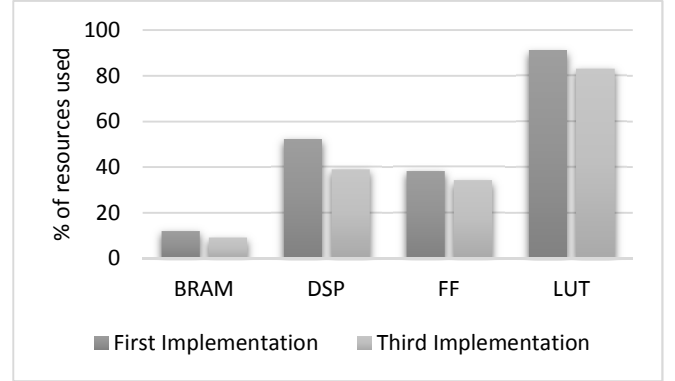


Fig. 8. The resources used by CNN-1 usign first and third implementation.

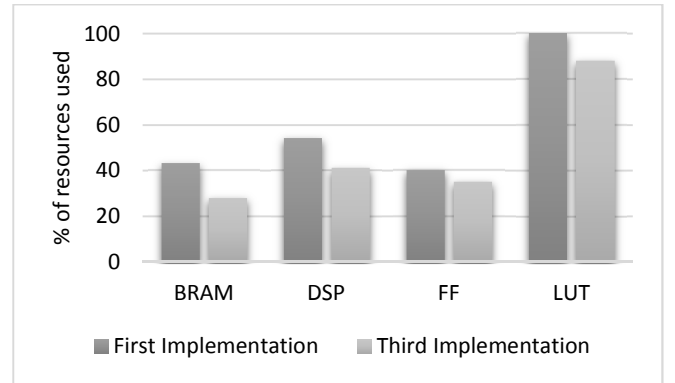


Fig. 9. The resources used by CNN-2 usign first and third implementation.

From Fig. 8 and Fig. 9, it is possible to verify that using the third implementation we can save 14.5% total resources to implement the CNN-1 and 19% total resources to implement the CNN-2.

C. Comparison CNN with the literature

In this section, the CNN described in table VI is compared with the CNN implementations found in the literature.

TABLE VI. DESCRIPTION OF TWO CNN USED FOR EXPERIMENTAL ASSESSMENT

Topology ^a	I[116×116@1]→C[12@5×5 STR 2] → P[12@2×2 STR 2] → C[9@5×5 STR 1] → P[9@2×2 STR 2] → C[11@3×3 STR 5] → F[10]
non-linearity in the hidden layers	ReLU
non-linearity in the output layer	Softmax

Fig. 10 presents resources used in the implementation of the CNN described in table VI. The latency of this design is approximately 1.5ms and the throughput is 2453 images of 116X116 dimensions per second.

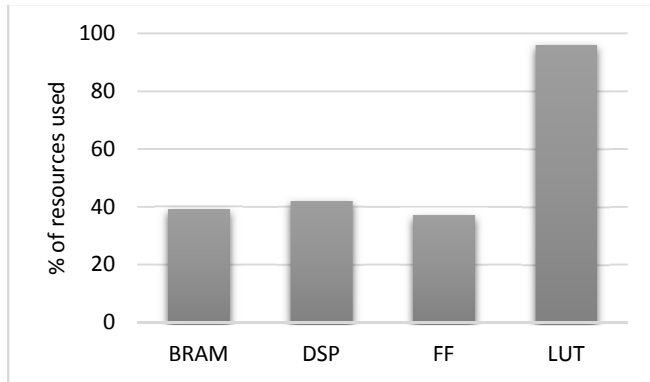


Fig. 10. FPGA resources used by the CNN describe in Table VI; LUT: Look-up Table; FF: Flip-Flops; BRAM: Block of Memory; DSP: Digital Signal Processor;

Table VII summarizes the implemented CNN system and compares it with the published results by other authors. Note that, since the different implementations use different FPGAs and exploit different CNN topologies and parallelism opportunities, a straightforward comparison between them is a hard task. While most of the solutions in the literature present implementations using fixed point precision [6]–[8], [24]. An implementation in floating-point is implemented herein offering a wider range of numbers and more precision than fixed point. However, a recent implementation using floating-point was developed [11]. This floating-point CNN was implemented using the HLS tool. However, that implementation focuses only in implementing convolution layers on an FPGA. An integration of other layers, such as max-pooling layer, and non-linear function was not studied. Also, in contrast with the presented implementation, solutions presented in the literature need an external memory in order to store/retrieve kernels and an on-/off-chip interconnect. Naturally this makes our solution use more internal resources.

TABLE VII. A COMPARISON WITH PUBLISH RESULTS

	FPGA	f (MHz)	Quantization	Throughput (GOPs)
[6]	Spartan3A DSP3400	125	Fixed	5.25
[6]	Virtex4 SX35	125	Fixed	5.25
[8]	Virtex5 SX240T	200	Fixed	16.0
[24]	Virtex5 LX330T	115	Fixed	6.74
[7]	Virtex5 SX240T	200	Fixed	7.0
[11]	Virtex7 VX485T	100	Float	61.62
Our	Kintex7 KC705	100	Float	12.63

VI. CONCLUSIONS

A platform to emulate a generic parameterizable-based deep learning solution on an FPGA is presented as an original way to implement a CNN system on an FPGA. Advantages of using this platform are to reduce the amount of code to be written, which saves time, and the implementation of a CNN system on an FPGA without the necessity of in-depth understanding of the underlying hardware. A designer can specify the topology through a configuration file, enabling the implementation of any CNN which adopts as layers: convolution, pooling and fully-connected. Through this platform, a CNN system is implemented on an FPGA using flow architecture. In this architecture, the entire system is seen as a series of input data transformations, where all layers are independent of each other.

The layers are commonly accompanied by a non-linearity: hyperbolic tangent or ReLU. Both yield strong practical implications. Currently, the ReLU is the most used because speed up the training process. In this paper another advantage of using ReLU is also presented, if its implementation on an FPGA is considered. It is possible save around 18% of the total resources if a CNN with a ReLU is used.

In this paper is also presented how save resources and keep the throughput of the entire CNN by removing the optimization directives in layers which presents lower latency. Using the implementation with best optimization resources is possible save 14.5% and 19% of the total resources in the CNN-1 and CNN-2, respectively.

ACKNOWLEDGMENT

This research has been supported by the Portuguese Foundation for Science and Technology through Projeto Estratégico UID/EEA/50009/2019, ARDITI - Agência Regional para o Desenvolvimento da Investigação, Tecnologia e Inovação under the scope of the Project M1420-09-5369-FSE-000001-PhD Studentship and MITIExcell - Excelencia Internacional de IDT&I NAS TIC (Project Number M1420-01-01450FEDER0000002), provided by the Regional Government of Madeira. It was also partially supported by Portuguese national funds through FCT (Fundação para a Ciência e a Tecnologia, Portugal) project UID/CEC/50021/2019.

REFERENCES

- [1] S. M. Trimberger, "Three ages of FPGAs: A retrospective on the first thirty years of FPGA technology," *Proc. IEEE*, vol. 103, no. 3, pp. 318–331, 2015.
- [2] E. Säking, B. E. Boser, and L. D. Jackel, "A neurocomputer board based on the ANNA neural network chip," in *Proceeding NIPS'91 Proceedings of the 4th International Conference on Neural Information Processing Systems*, 1991, pp. 773–780.
- [3] E. Sackinger, B. E. Boser, J. Bromley, Y. LeCun, and L. D. Jackel, "Application of the ANNA Neural Network Chip to High-Speed Character Recognition," *IEEE Trans. Neural Networks*, vol. 3,

no. 3, pp. 498–505, 1992.

- [4] E. Säcker and H. P. Graf, “A system for high-speed pattern recognition and image analysis,” in *Proceedings of the Fourth International Conference on Microelectronics for Neural Networks and Fuzzy Systems*, 1994.
- [5] J. Cloutier, E. Cosatto, S. Pigeon, F. R. Boyer, and P. Y. Simard, “VIP: an FPGA-based processor for image processing and neural networks,” *Proc. Fifth Int. Conf. Microelectron. Neural Networks*, pp. 330–336, 1996.
- [6] C. Farabet, C. Poulet, J. Y. Han, and Y. LeCun, “CNP: An FPGA-based processor for Convolutional Networks,” in *FPL 09: 19th International Conference on Field Programmable Logic and Applications*, 2009, pp. 32–37.
- [7] S. Cadambi, A. Majumdar, M. Becchi, S. Chakradhar, and H. P. Graf, “A programmable parallel accelerator for learning and classification,” in *Proceedings of the 19th international conference on Parallel architectures and compilation techniques - PACT '10*, 2010, p. 273.
- [8] S. Chakradhar, M. Sankaradas, V. Jakkula, and S. Cadambi, “A dynamically configurable coprocessor for convolutional neural networks,” *ACM SIGARCH Comput. Archit. News*, vol. 38, no. 3, p. 247, 2010.
- [9] M. Sankaradas *et al.*, “A Massively Parallel Coprocessor for Convolutional Neural Networks,” *2009 20th IEEE Int. Conf. Appl. Syst. Archit. Process.*, pp. 53–60, 2009.
- [10] A. Dunder *et al.*, “Accelerating Deep Neural Networks on Mobile Processor with Embedded Programmable Logic,” *Proc. NIPS'13*, 2013.
- [11] C. Zhang, P. Li, G. Sun, Y. Guan, B. Xiao, and J. Cong, “Optimizing FPGA-based Accelerator Design for Deep Convolutional Neural Networks,” in *Proceedings of the 2015 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays - FPGA '15*, 2015, pp. 161–170.
- [12] W. Meeus, K. Van Beeck, T. Goedemé, J. Meel, and D. Stroobandt, “An overview of today’s high-level synthesis tools,” *Design Automation for Embedded Systems*. 2012.
- [13] W. H. and L. D. J. Y. Le Cun, B. Boser, J. S. Denker, D. Henderson, R.E. Howard, “Handwritten Digit Recognition with a Back-Propagation Network,” *Adv. Neural Inf. Process. Syst.*, pp. 396–404, 1990.
- [14] L. D. LeCun, Y.;Boser, B.; Denker, J. S.; Henderson, D.; Howard, R. E.; Hubbard, W.; Jackel, “Backpropagation Applied to Handwritten Zip Code Recognition,” *Neural Computation*, vol. 1, no. 4. pp. 541–551, 1989.
- [15] Y. LeCun and L. Bottou, “Sn: A simulator for connectionist models,” in *Proceedings of NeuroNimes 88*, 1988.
- [16] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” *Adv. Neural Inf. Process. Syst.*, pp. 1–9, 2012.
- [17] K. Simonyan and A. Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,” in *ICLR 2015*, 2015.
- [18] C. Szegedy *et al.*, “Going deeper with convolutions,” in *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, 2015.
- [19] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” *arxiv.org*, 2015.
- [20] A. Goodfellow, Ian; Bengio, Yoshua; Courville, *Deep Learning*. Cambridge, MA: MIT Press, 2016.
- [21] J. Nagi and F. Ducatelle, “Max-pooling convolutional neural networks for vision-based hand gesture recognition,” *2011 IEEE Int. Conf. Signal Image Process. Appl.*, pp. 342–347, 2011.
- [22] R. Memisevic and C. Zach, “Gated softmax classification,” *Adv. Neural Inf. Process. Syst. 23 (NIPS 2010)*, pp. 1–9, 2011.
- [23] Ivo Nascimento, Ricardo Jardim, and F. Morgado-Dias, “Hyperbolic tangent implementation in hardware: a new solution using polynomial modeling of the fractional exponential part,” *Neural Comput. Appl.*, vol. 23, no. 2, pp. 363–369, 2013.
- [24] M. Sankaradas *et al.*, “A Massively Parallel Coprocessor for Convolutional Neural Networks,” in *2009 20th IEEE International Conference on Application-specific Systems, Architectures and Processors*, 2009, pp. 53–60.